

Le langage C

François Trahay & Gaël Thomas



Présentation du module

Objectifs du module:

- Maîtriser le langage C
- Savoir s'adresser au système d'exploitation depuis un programme

Modalités:

- Un peu de théorie
- Beaucoup de pratique

Contenu du module

Partie *Programmation*

- CI 1 – Le langage C
- CI 2 – Faire des programmes modulaires en C
 - Début du projet
- CI 3 – Les pointeurs
- CI 4 – Debugger un programme

Partie *Système*

- CI 5 – Les fichiers
- CI 6 – Les processus
- CI 7 – Appels système et Sémaphores
- CI 8 – Signaux

Evaluation

- CI 9 – Travail sur le projet
- CI 10 – Finalisation du projet et oral
- CF1 – Examen sur papier

Evaluation du module

Le module est évalué par

- un projet
 - but: que les étudiants programment
 - risque: certains étudiants peuvent “être aidés” (par un camarade/une IA)
 - évaluation par un oral/revue de code de 15 minutes.
 - Exemple de questions:
 - *Comment as-tu implémenté la fonction `foo` ?*
 - *Que se passe-t-il si la fonction `bar(int n)` est appelée avec $n < 0$?*
- un examen sur papier
 - but: évaluer la capacité des étudiants à résoudre un problème en C

Déroulement d'une séance

Système de *classe inversée*. Pour chaque séance :

- **Avant** la séance
 - Etude de la partie cours (y compris les commentaires des slides) de la séance à venir
- **Pendant** la séance:
 - Mini-évaluation de la partie cours (Kahoot!)
 - Explications sur les points mal compris
 - Travaux pratiques : expérimentations sur les concepts vus en cours

Attention ! Cela ne fonctionne que si vous travaillez sérieusement **avant** la séance.

Hypothèse: les étudiants suivant ce cours sont des adultes responsables.

Ressources disponibles

Pour vous aider, vous avez à votre disposition:

- Le poly contenant l'ensemble des transparents commentés
- Les transparents en version pdf/html
 - en html, appuyez sur S pour afficher les commentaires
- La documentation des fonctions C standard (`man 2 <fonction>` ou `man 3 <fonction>`)
- Une équipe enseignante de choc !

Conseils pour la suite de votre scolarité

- N'utilisez pas d'IA pour vos TP/projets
 - ChatGPT / Copilot sont des outils pour **améliorer la productivité** des développeurs
 - Les TP servent à **apprendre**, pas à produire
- Ne consultez les corrigés des TP qu'en dernier recours
 - Essayez de faire les exercices
 - Posez des questions aux enseignants

C vs. Java

- langage de *bas niveau* vs. *haut niveau*
- En C, manipulation de la mémoire et de ressources proches du matériel
- “*Un grand pouvoir implique de grandes responsabilités*”¹
- programmation impérative vs. programmation objet

¹ B. Parker, *Amazing Fantasy*, 1962

Mon premier programme en C

Fichier *.c

```
/* hello_world.c */
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
    printf("Hello World!\n");
    return EXIT_SUCCESS;
}
```

Compilation/execution:

```
$ gcc hello_world.c -o hello_world -Wall -Werror
$ ./hello_world
Hello World!
```

Déclaration de variable

Pour les types simples, déclaration identique à Java:

```
int var1;  
int var2, var3, var4;  
int var5 = 42;
```

Types disponibles:

- pour les entiers: `int`, `short`, `long`, `long long`
- pour les flottants: `float`, `double`
- pour les caractères: `char`

Portée des variables

Une variable est accessible dans le *bloc* dans laquelle elle est déclarée

```
void foo() {  
    if(x==0) {  
        int a = 17;  
        for(int i=0; i< 10; i++) {  
            int a = i;  
            printf(" a=%d\n", a); // affiche a=0, a=1, ... a=9  
        }  
        printf("a=%d\n", a); // affiche a=17  
    }  
    printf("a=%d\n", a); // erreur de compilation: a n'existe pas  
}
```

Une **variable globale** est déclarée en dehors de tout bloc. Elle est accessible depuis tout le fichier.

Opérateurs et Expressions

La liste des opérateurs disponibles est à peu près la même qu'en Java:

- arithmétique : +, -, *, /, %
- affectation : =, +=, -=, *=, /=, % =
- incrémentation/décrémentation: ++, --
- comparaison: <, <=, >, >=, ==, !=
- logique: !, &&, ||

Mais également:

- `sizeof n`: donne le nombre d'octets qui constitue une variable/un type n

Opérateurs bit à bit

Possibilité de travailler sur des *champs de bits*.

- Opération sur les bits d'une variable
- décalage: <<, >>
- OR : |, AND : &, XOR : ^, NOT : ~
- affectation: <<=, >>=, |=, &=, ^=, ~=

Structures algorithmiques

Comme en Java:

- `for(i=0; i<n; i++) { ... }`
- `while(cond) {... }`
- `do { ... } while(cond);`
- `if (cond) { ... } else { ... }`
- `break` pour sortir d'une boucle

Affichage / Lecture

- Pour afficher: `printf("%d exemple de %f format \n", v1, v2);`
- Pour lire: `scanf("%d-%f", &v1, &v2);`

Fonctions

Déclaration:

```
type_retour nom_fonc(type_param1 param1, type_param2 param2) {  
    /* déclaration des variables locales */  
    /* instructions à exécuter */  
}
```


Du type primitif au type composé

- Jusqu'à maintenant, vous avez vu les types primitifs
 - `char`, `short`, `int`, `long` `long` (signés par défaut, non signés si préfixés de `unsigned`)
 - `float` (4 octets), `double` (8 octets)
- Dans ce cours, nous apprenons à définir de nouveaux types de données
 - Une **structure** est constituée de sous-types **hétérogènes**
 - Un **tableau** est constitué de sous-types **homogènes**

Les structures

Une structure est une définition d'un nouveau type de données

- composé de sous-types nommés (primitifs ou composés)
- possédant un nom

Remarque: les sous-types d'une structure s'appellent des champs

Définition d'une nouvelle structure avec :

```
struct nom_de_la_structure {  
    type1 nom_du_champs1;  
    type2 nom_du_champs2;  
    ...  
};
```

Par exemple:

```
struct nombre_complexe {  
    int partie_reelle;  
    int partie_imaginaire;  
};
```

Par convention, les noms de structures commencent par une minuscule en C

Déclaration d'une variable de type structure

- Une déclaration d'une variable de type structure se fait comme avec un type primitif:

```
struct nombre_complexe z1, z2, z3;
```

- On peut aussi initialiser les champs de la structure lors de la déclaration:

```
/* partie_relle de z prend la valeur 0 */  
/* partie_imaginaire de z prend la valeur 1 */  
struct nombre_complexe i = { 0, 1 };  
/* autre solution : */  
struct nombre_complexe j = { .partie_reelle=0, .partie_imaginaire=1 };
```

Accès aux champs d'une variable de type structure

- L'accès aux champs d'une variable de type structure se fait en donnant le nom de la variable, suivi d'un point, suivi du nom du champs :

```
struct point {  
    int x;  
    int y;  
};  
  
struct ligne {  
    struct point p1;  
    struct point p2;  
};
```

```
void f() {  
    struct point p;  
    struct ligne l;  
  
    p.x = 42;  
    p.y = 17;  
    l.p1.x = 1;  
    l.p1.y = 2;  
    l.p2 = p; /* copie p.x/p.y dans l.p2.x/l.p2.y */  
  
    printf("[%d %d]\n", p.x, p.y);  
}
```

Les tableaux

- Un tableau est un type de données composé de sous-types homogènes
 - Les éléments d'un tableau peuvent être de n'importe quel type (primitif, structure, mais aussi tableau)
 - Pour déclarer un tableau:

```
type_des_elements nom_de_la_variable[taille_du_tableau];
```

Par exemple:

```
int      a[5];      /* tableau de 5 entiers */
double   b[12];     /* tableau de 12 nombres flottants */
struct point c[10]; /* tableau de 10 structures points */
int      d[12][10]; /* tableau de 10 tableaux de 12 entiers */
           /* => d est une matrice 12x10 */
```

Accès aux éléments d'un tableau

- L'accès à l'élément `n` du tableau `tab` se fait avec `tab[n]`
- Un tableau est indexé à partir de zéro (éléments vont de 0 à `N - 1`)

```
void f() {  
    int x[3];  
    int y[3];  
    int i;  
  
    /* 0 est le premier élément, 2 est le dernier */  
    for(i=0; i<3; i++) {  
        x[i] = i;  
        y[i] = x[i] * 2;  
    }  
}
```

Tableaux et structures

- On peut mixer les tableaux et les structures, par exemple :

```
struct point {  
    int x;  
    int y;  
};  
  
struct triangle {  
    struct point sommets[3];  
};
```

```
void f() {  
    struct triangle t;  
  
    for(i=0; i<3; i++) {  
        t.sommets[i].x = i;  
        t.sommets[i].y = i * 2;  
    }  
}
```

Différences par rapport à Java

- On ne peut pas accéder à la taille d'un tableau
- Lors d'un accès en dehors des bornes du tableau, l'erreur est silencieuse : c'est une erreur, mais elle n'est pas signalée immédiatement
=> parmi les erreurs les plus fréquentes (et les plus difficiles à repérer) en C

```
void f() {  
    int x[3];  
  
    x[4] = 42; /* Erreur silencieuse !!! */  
              /* Écriture à un emplacement aléatoire en mémoire */  
              /* le bug pourra apparaître n'importe quand */  
}
```


Initialisation d'un tableau lors de sa déclaration

- Un tableau peut être initialisé lorsqu'il est déclaré avec

```
type_element nom_variable[taille] = { e0, e1, e2, ... };
```

- Par exemple: `int x[6] = { 1, 2, 3, 4, 5, 6 };`
- Comme pour les structures, on peut partiellement initialiser un tableau
 - Par exemple: `int x[6] = { 1, 1, 1 };`

Initialisation mixte de tableaux et structures

- On peut composer des initialisations de tableaux et de structures

```
struct point {  
    int x;  
    int y;  
};  
  
struct triangle {  
    struct point sommets[3];  
};  
  
struct triangle t = {  
    { 1, 1 },  
    { 2, 3 },  
    { 4, 9 }  
};
```

Tableaux et chaînes de caractères

Une chaîne de caractère est simplement un tableau de caractères terminé par le caractère '`\0`' (c'est à dire le nombre zéro)

```
char yes[] = "yes";
```

est équivalent à

```
char yes[] = { 'y', 'e', 's', '\0' };
```

Typedef

L'instruction `typedef` permet de donner un nom à type:

```
typedef int entier;
entier i=5; // équivalent à int i=5;

typedef struct node node_t;
node_t n; // déclare un struct node

typedef char[16] name;

// définit à la fois le type 'struct complexe' et le type 'complexe'
typedef struct complexe { int partie_reelle; int partie_imaginaire } complexe_t;
```

Passage par valeur et par référence

- En C, il existe deux types de passage d'arguments :
 - Passage **par valeur** : l'argument est copiée de l'appelé vers l'appelant
=> l'argument et sa copie sont deux variables différentes
 - Passage **par référence** : une référence vers l'argument de l'appelant est donné à l'appelé
=> l'appelant et l'appelé partagent la même donnée
- Par défaut :
 - Les **tableaux** sont passés par **référence**
 - Un argument de type tableau est déclaré avec `type nom[ ]`, sans la taille
 - Les **autres types** sont passés par **valeur**

Passage par valeur – les types primitifs

```
/* le x de f et le x du main sont deux variables distinctes */  
/* le fait qu'elles aient le même nom est anecdotique */  
void f(int x) {  
    x = 666;  
    printf("f : x = %d\n", x);          /* f : x = 666 */  
}  
  
int main() {  
    int x = 42;  
    f(x);                             /* x est copié dans f */  
    /* => le x de main n'est donc pas modifié par f */  
    printf("g : x = %d\n", x);          /* g : x = 42 */  
    return 0;  
}
```

Passage par valeur – les structures

```
struct point {  
    int x;  
    int y;  
};  
  
void f(struct point p) {  
    p.x = 1;  
    printf("(%d, %d)\n", p.x, p.y);    /* => (1, 2) */  
}  
  
int main() {  
    struct point p = { -2, 2 };  
    f(p);                               /* p est copié dans f */  
    printf("(%d, %d)\n", p.x, p.y);    /* => (-2, 2) */  
    return 0;  
}
```

Passage par référence – les tableaux

```
void print(int x[], int n) {  
    for(int i=0; i<n; i++) {  
        printf("%d ", x[i]);  
    }  
    printf("\n");  
}
```

```
int main() {  
    int tab[] = { 1, 2, 3 };  
  
    print(tab, 3); /* => 1 2 3 */  
    f(tab);  
    print(tab, 3); /* => 1 42 3 */  
  
    return 0;  
}
```

```
/* x est une référence vers le tableau original */  
void f(int x[]) {  
    x[1] = 42;          /* => modifie l'original */  
}
```


